

A Framework for Designing Reward Functions: From Objectives to Features to Human-Aligned Reward Functions

Di Yang Shi¹, W. Bradley Knox¹

{dyshi, bradknox}@cs.utexas.edu

¹University of Texas at Austin

Abstract

We present a formal process to enable non-experts to instantiate and iterate on human-aligned reward functions, i.e. reward functions that adhere to a given preference ordering over trajectories. Given a task described in natural language, our process produces a linear reward function in three steps: distill the task’s objectives into a set of fundamental objectives and derive measurable outcome variables that capture those fundamental objectives, select a causally representative subset of outcome variables as the reward terms, and fit weights to those reward terms via preference elicitation. Our contributions describe the first step and formalize the latter two steps. The first is a guided workflow for deriving outcome variables. The second is a reduction of reward term selection to minimum-cost partial cover on a causal DAG, solved in polynomial time via max-flow. The third is a geometric framing of weight fitting as a convex feasibility problem iteratively narrowed by preference queries, solved by existing separation oracle methods. To the best of our knowledge, this is the first reward-design method that maintains a deterministically conflict-free feasible weight region, narrowed to a desired tolerance via a separation oracle with $O(n \log \kappa)$ preference queries.

1 Introduction

In reinforcement learning (RL) the design of a human-aligned reward function—one that faithfully reflects intended preferences over trajectories—is fundamental to obtaining policies that behave as desired, yet remains a process largely restricted to experts. Even with expert RL practitioners, reward functions are commonly derived through trial-and-error processes (Knox et al., 2023; Booth et al., 2023) and still fall victim to common pitfalls such as reward hacking, redundancy in reward terms, and mismatched preferences in behavior from those of humans (Knox et al., 2023).

We propose a framework (Figure 1) that takes as input a task described in natural language and produces a human-aligned linear reward function. The framework consists of three steps: (i) distill the task description into a set of fundamental objectives and derive measurable outcome variables that capture those fundamental objectives; (ii) select a causally representative, low-cost subset of outcome variables as the reward terms; and (iii) fit weights to those reward terms via preference elicitation.

Our contributions are the specification of these three steps in detail, including formalization of the last two steps. The first contribution (Section 3.1) is a guided workflow for distilling fundamental objectives from an initial task description and deriving measurable outcome variables from them. The second (Section 3.2) assumes that causal relationships between outcome variables are determinable and formalizes the selection of a low-cost, causally representative subset of these variables

as a minimum-cost partial cover problem on a DAG, which we solve via reduction to max-flow. This turns a heuristically driven manual process with deduplication inefficiencies to a more formal, algorithmic approach that is optimal by design. The third contribution (Section 3.3) fits weights to the selected reward terms. [Dulac-Arnold et al. \(2019\)](#) consider multi-dimensional rewards to be one of the primary challenges in RL, particularly in how to balance tradeoffs between reward terms representing different objectives. We address this problem with an algorithmic approach that precisely queries how these tradeoffs should be made. Reframing the weight determination as a convex optimization problem in narrowing down a feasible weight space, we leverage an existing optimization method to analytically determine the best trajectory pairs to query. Such pairs correspond to a hyperplane that cuts the feasible region in half. Accordingly, our reframed approach inherits asymptotically optimal preference querying efficiency and likely-optimal runtime ([Jiang et al., 2020](#)).

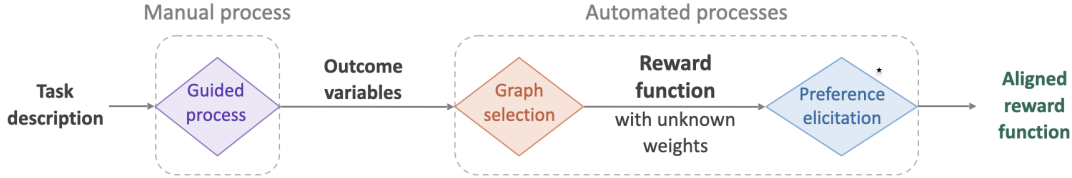


Figure 1: An outline of our process with the colored diamonds as our contributions (sections 3.1, 3.2, and 3.3 respectively). The “preference elicitation” node has the caveat that it is dependent on an oracle which may be a human expert.

2 Related Work

We organize related work by the step of our framework that it addresses.

Objective elicitation and outcome variable derivation. Our approach to selecting fundamental objectives, a rather subjective and contextual problem, is inspired by [Keeney \(1992\)](#)’s work which advocates for first focusing on the values we strive for rather than the alternatives which are available to us and then exploring options based upon those defined values.

Reward term selection and representation. A common approach to alignment is reward shaping, the inclusion of additional reward terms towards shaping intermediary learning of policy behavior ([Ng et al., 1999](#); [Wiewiora et al., 2003](#)). However, these approaches tend to fall prey to [Sutton \(2019\)](#)’s bitter lesson, leaking biases towards how humans think a task should be done rather than motivating the outcomes we intend. Recent empirical studies have highlighted these shortcomings in designing reward functions even for experienced RL practitioners ([Knox et al., 2023](#); [Booth et al., 2023](#)).

Weight fitting via preference elicitation. Reinforcement learning from human feedback (RLHF) has become popularized ([Christiano et al., 2017](#); [Ouyang et al., 2022](#)) but the querying process is typically Bayesian or unspecified altogether. In contrast, our querying method is deterministic with concrete bounds. A paper under review by [Hatgis-Kessell \(2026\)](#) explores interactive LLM-facilitated reward design with a human preference annotator; we hope to deploy our method with a similar medium but focus solely on our method’s process and theory in this work. Our work is perhaps closest to [Sadigh et al. \(2017\)](#)’s setting in active preference learning in that it uses active preference querying using synthesized trajectories and predicates efficiency on volume removal of the feasible space. However, instead of assuming a feature function we offer a process to build one that’s designed towards addressing aforementioned issues such as redundancy in reward terms and mismatch in preferences between the policy and what’s intended. We also employ analytic approaches to calculate optimal cutting planes rather than sampling multiple cuts for empirical estimates of volume removal over a distribution. This further allows us to output definitive weights rather than probabilistic ones.

3 Method

This section presents our three contributions, which correspond to the labeled steps in Figure 1. Section 3.1 derives a set of candidate outcome variables from a task description. Section 3.2 selects a causally representative, low-cost subset of those candidates as the reward terms. Section 3.3 fits weights to the resulting linear reward representation, producing a polytope of weight vectors consistent with elicited preferences.

3.1 Selecting Outcome Variables from Task Objectives

This section derives a set of candidate outcome variables from a task description in two steps. Section 3.1.1 distills the task’s initial objectives into a set of fundamental objectives. Section 3.1.2 then derives measurable outcome variables that serve as proxies for those fundamental objectives. The resulting candidate set is filtered in Section 3.2.

3.1.1 Distilling Task Objectives Into Fundamental Objectives

This step takes a natural-language description of the task as input and produces a set of fundamental objectives via an iterative procedure. For a given task, we start by exhaustively listing objectives of the task. Then, we scrutinize each objective in this list asking why each one is important. The answers may elucidate more objectives, progressing towards more fundamental ones with each iteration. We repeat this process of iteratively conceptualizing task objectives and distilling them into more fundamental ones until the fundamental objectives satisfactorily cover the task objectives. It may be the case that an initial task objective is already fundamental and can be left as-is. We provide some examples in Table 1.

Initial Objectives	Fundamental Objectives
avoid traffic, avoid construction	minimize time
avoid tolls, take fuel efficient routes	minimize cost
drive safely, drive smoothly	minimize collisions, minimize passenger discomfort

Table 1: Example fundamental objectives derived from initial objectives

3.1.2 Deriving Measurable Outcome Variables from Objectives

Fundamental objectives such as “minimize passenger discomfort” or “maximize happiness” are typically not directly measurable, so we cannot reward the agent for them as stated. We therefore introduce measurable proxies that we call *outcome variables*. In this subsection we produce a candidate set of outcome variables, with one or more per fundamental objective.

We say that a set of outcome variables *captures* a fundamental objective if there is some operation on the output of their sampling which sufficiently approximates our alignment to a fundamental objective. For each fundamental objective, we assume that there is some proxy set of measurable outcome variables that capture it; if this is not the case, one may consider redefining the fundamental objectives. We show how the previous fundamental objectives might translate to outcome variables in Table 2 and describe some possible issues and potential remedies in Table 3.

Fundamental Objectives	Outcome Variables
minimize time	trip duration(seconds)
minimize cost	trip cost(\$)
minimize collisions	peak acceleration / jerk (m/s^2 or m/s^3)
minimize passenger discomfort	passenger satisfaction(scalar value)

Table 2: Converting fundamental objectives to measurable outcome variables

Potential Issues	Remedy
No measurable proxy exists	Decompose the objective into more concrete sub-objectives for which proxies exist.
Proxies not observable during training	Substitute with causally upstream variables observable at training time.
Proxies are gameable	Diversify the proxy set so that simultaneously gaming all proxies requires behavior that approximates the fundamental objective.

Table 3: Shortcomings and remedies in identifying measurable proxies for fundamental objectives.

3.2 A Causally-Aware Outcome Variable Selection Framework

The candidate set produced by Section 3.1 typically contains more outcome variables than we want to include in the reward function. Outcome variables have some cost (selection, measurement, quality, etc.), and many candidates are redundant under causality: a causally downstream variable reflects the effects of its upstream variables, so measuring the upstream variable can substitute for measuring the downstream ones. This subsection formalizes the resulting selection problem on the causal DAG of outcome variables, presents an exact algorithm via reduction to max-flow, and gives a locally greedy alternative for cases where global cost or causal information is incomplete.

We propose the selection process of outcome variables as a graph algorithm. We denote a *causal relationship* between two nodes a, b as $a \rightarrow b$ if a has a causal effect on b . We assume causal relationships are determinable; depending on the domain, these causal relationships may be provided by human experts or possibly even automated. Let O be a set of outcome variables representing the fundamental objectives we derived and C be the set of causal relationships between them. In general, causality cannot be cyclical but this fact does not preclude feedback cycles in causality through a temporal dimension. Thus, we assert that C is *temporally-fixed*, that is, for any subset of C which forms a cycle which necessarily transcends time we effectively cut that cycle and fix a temporal segment to consider. In practice, these chicken-and-egg decisions depend on what outcome variables are first measurable in the task at hand. For example, both minimizing trip time and minimizing collisions causally influence each other; when a cycle like this arises, we fix a temporal ordering based on what is first measurable (here, trip time), eliminating the reverse edge.

3.2.1 Problem Definition

We formalize the selection problem as a minimum-cost cover on the causal DAG of outcome variables, where the demand nodes are the candidates we wish to account for in the reward.

Let $V = O$, $E = C$, and $G = (V, E)$ be a directed acyclic graph with node costs $c : V \rightarrow \mathbb{Q}_{\geq 0}$, $D \subseteq V$ be a set of *demand nodes*, and $\text{src}(G)$ denote the set of source nodes of G (i.e., nodes with in-degree zero). A set $S \subseteq V$ is a *valid cover* of D if every node in D is covered, where coverage is defined recursively as follows.

Definition 1 (Coverage). *Given a selection $S \subseteq V$, a node $v \in V$ is covered by S if either $v \in S$, or $v \notin \text{src}(G)$ and every parent of v is covered by S .*

The **Minimum Cost Partial Cover** (MCPC) problem asks for a valid cover S^* minimizing $\sum_{v \in S^*} c(v)$. We show that the MCPC problem can be reduced to the max-flow min-cut (Ford & Fulkerson, 1956) problem in [Appendix: Reduction of Minimum Cost Partial Cover](#).

3.2.2 Reduction to Minimum Cut

Reducing MCPC to a standard s - t flow problem yields polynomial-time solvability via well-known max-flow algorithms. By Theorem 2, the MCPC equals the minimum weight vertex cut separating all sources from all demand nodes. We reduce this to a standard minimum s - t cut problem via node splitting. We detail the construction of the MCPC problem as a flow network below.

Definition 2 (Minimum Cost Partial Cover Flow Network). *Given $G = (V, E)$, costs c , and demand set D , construct a flow network $\mathcal{N} = (V', E', \sigma, \tau)$ as follows.*

- For each node $v \in V$: add nodes v^{in} and v^{out} with a directed edge $v^{in} \rightarrow v^{out}$ of capacity $c(v)$.
- For each edge $(u, v) \in E$: add a directed edge $u^{out} \rightarrow v^{in}$ of capacity ∞ .
- Add a supersource σ and for each source $r \in \text{src}(G)$: add edge $\sigma \rightarrow r^{in}$ of capacity ∞ .
- Add a supersink τ and for each demand node $d \in D$: add edge $d^{out} \rightarrow \tau$ of capacity ∞ .

3.2.3 Algorithm

We now present [Algorithm 1](#) for selecting a causal cover of outcome variables from a DAG of their causal relationships.

Algorithm 1 Minimum Cost Partial Cover As Max Flow Min Cut

Require: DAG $G = (V, E)$, costs $c : V \rightarrow \mathbb{Q}_{\geq 0}$, demand set $D \subseteq V$

Ensure: Minimum cost set $S^* \subseteq V$ covering all nodes in D

- 1: Construct flow network $\mathcal{N}$ as in Definition 2
 - 2: Compute maximum σ - τ flow f^* in $\mathcal{N}$ using a max-flow algorithm
 - 3: Let R be the set of nodes reachable from σ in the residual graph of f^*
 - 4: **return** $S^* \leftarrow \{v \in V : v^{in} \in R \text{ and } v^{out} \notin R\}$
-

3.2.4 Runtime Complexity

The runtime is bound by the max-flow procedure. Two well-known methods are Edmonds-Karp (Edmonds & Karp, 1972), with complexity $O(|V'| |E'|^2)$, and Dinic's algorithm (Dinic, 1970), with complexity $O(|V'|^2 |E'|)$, where $|V'| = 2|V| + 2$ and $|E'| = |V| + |E| + |\text{src}(G)| + |D|$.

3.2.5 Locally Greedy Approach

In practice, the global structure assumed by the exact algorithm may not be available; the cost $c(v)$ of a node may not reduce to a single comparable metric or may be unknown, the causal relationships between some pairs may be unclear, or a practitioner may already have a proposed cover S in mind. In such cases, rather than solving for the globally optimal S^* , one may instead verify and refine S relative to a cost budget B .

1. Select a subset S of demand nodes that best represents the intended learning objectives. One may begin with $S = D$ if no expert selection is available.
2. If $\sum_{v \in S} c(v) \leq B$, the proposed cover is within budget and we are done. Otherwise, for each $s \in S$, traverse upward through the ancestors of s in G , replacing costly nodes in S with lower-cost ancestors where possible. We note this is a heuristic: the ancestor sets of distinct nodes in S may overlap, causing costs to be double-counted relative to the optimal S^* .
3. If S does not provide sufficient learning signal after refinement, additional reward shaping terms may be introduced from higher in G , subject to the remaining budget $B - \sum_{v \in S} c(v)$.

3.3 Fitting the Reward Function’s Parameters to Preferences

Section 3.2 produced a set of reward terms. To complete the reward function we need to (a) decide how those terms combine into a scalar reward and (b) fit the parameters of that combination. We assume a linear combination and justify this briefly below before describing the weight-fitting procedure. In their seminal work on inverse reinforcement learning, [Ng & Russell \(2000\)](#) argued that linear reward functions are necessary for computational feasibility, are generally sufficient, cover a representative subset of states, and support constraint relaxation with penalty terms—with nonlinearities captured in the basis functions instead. Linear reward functions also offer interpretability and enable core techniques such as successor features ([Barreto et al., 2017](#)); critically, our approach relies on convex optimization concepts (half-spaces, analytic centers, feasible regions) that are only sensible in a linear setting.

We then fit the weights by iteratively shrinking a feasible region in weight space using preference queries. Section 3.3.1 sets up the geometry of this feasible region, and Section 3.3.2 presents the synthetic-trajectory algorithm that proposes preference queries, along with its query complexity and runtime. The output is a polytope of weight vectors consistent with all elicited preferences.

3.3.1 Geometric Problem Description

We now describe the geometry that allows us to recast weight fitting as a convex feasibility problem solvable by separation oracle methods.

Let PROPOSE be the function that proposes trajectory pairs for preference evaluation, and DECIDE be the preference decision function (a human expert, algorithm, oracle, etc.). Given outcome variables selected as reward terms that capture the entire trajectory $x_1, \dots, x_n$, our goal is to recover the weights $w_1, \dots, w_n$ that best approximate DECIDE, defining a target reward function $R^*(x) = w^{*\top} x \in \mathbb{R}$, where $w^* = (w_1^*, \dots, w_n^*)$ is the true weight vector and $x = (x_1, \dots, x_n)$ is the feature vector for a trajectory. Because the reward is scale-invariant, only the direction of the weight vector matters; we identify the weight vector with a point on $S^{n-1} = \{w \in \mathbb{R}^n : \|w\| = 1\}$, reducing the problem to $n - 1$ degrees of freedom.

Each call $\text{PROPOSE}(\mathcal{C})$ yields an inequality constraint corresponding to a hyperplane through the origin. Concretely, PROPOSE selects a trajectory pair (τ^+, τ^-) with feature vectors $x(\tau^+), x(\tau^-) \in \mathbb{R}^n$; the induced query normal is $q = \Delta x / \|\Delta x\|$ where $\Delta x = x(\tau^+) - x(\tau^-)$. Once DECIDE returns sign $s \in \{-1, +1\}$, the resulting half-space constraint on the weight vector is $\{w : s q^\top w \geq 0\}$. Geometrically, we iteratively refine a convex polyhedral cone, projected onto S^{n-1} , to localise w^* . Let $\mathcal{C} \subset \mathcal{H}^n$ denote the set of accumulated constraints where $\mathcal{H}^n$ is the set of n -dimensional half-spaces, and $\kappa = nR/\epsilon$ where R is the radius of the initial containing box and ϵ is the desired error tolerance.

3.3.2 Synthetic Trajectory Algorithms

Our primary approach narrows $\mathcal{C}$ via $\mathcal{C} \leftarrow \mathcal{C} \cup \{\text{DECIDE}(\text{PROPOSE}(\mathcal{C}))\}$. Trajectory pairs submitted for preference queries are constructed entirely from scratch rather than sampled from a fixed dataset, enabling us to pose arbitrary hypothetical comparisons—analogueous to asking “would you rather” questions when eliciting another person’s values.

This framework is equivalent to the *separation oracle* (SO) setting of [Grötschel et al. \(1981\)](#). The most prominent SO method is the analytic center cutting plane method (ACCPM) ([Atkinson & Vaidya, 1995](#)); the most asymptotically efficient is the volumetric center method ([Vaidya, 1996](#)) (with computational improvements by [Jiang et al. 2020](#)). Both use $O(n \log \kappa)$ oracle calls and differ primarily in how they select the query point: ACCPM uses a distance-based criterion (the analytic center), while the volumetric center uses a volume-based criterion. As an implementation note, we substitute the positive unit hypercube as the initial feasible region and normalize output weights as needed; this implicitly restricts $w \geq 0$, which assumes outcome variables are oriented so that larger

values are preferred—if negative weights are required, a standard hypercube $[-1, 1]^n$ should be used instead.

In practice, we expect n to be small, so ACCPM is sufficient in most cases; when oracle calls are prohibitively expensive, the volumetric center method is preferred as it achieves greater volume reduction per call (Vaidya, 1996). Once the query point is determined by the center computation (see Table 4), constructing the half-space constraint $\{w : s q^\top w \geq 0\}$ and synthesizing a corresponding trajectory pair (τ^+, τ^-) with $x(\tau^+) - x(\tau^-) = \lambda q$, where $\lambda > 0$ is chosen to produce outcome magnitudes that are meaningful to the evaluator, requires $O(n)$ additional work. Table 4 summarizes the two methods.

Method	# Oracle Calls	Runtime	Barrier Function
Analytic Center (Atkinson & Vaidya, 1995)	$O(n \log \kappa)$	$O(n^{\omega+1} \log^2 \kappa + (n \log \kappa)^{2+\omega/2})$	$-\sum_{i=1}^m \log(b_i - a_i^T x)$
Volumetric Center (Vaidya, 1996)	$O(n \log \kappa)$	$O(n^3 \log \kappa)$ (Jiang et al., 2020)	$-\log \det \sum_{i=1}^m \frac{a_i a_i^T}{(b_i - a_i^T x)^2}$

n : dimension; $\kappa = nR/\epsilon$; ϵ : error tolerance; R : radius of initial containing box; ω : matrix multiplication exponent; $m = |\mathcal{C}|$

Table 4: Comparison of ACCPM (Atkinson & Vaidya, 1995) and the volumetric center method (Vaidya, 1996; Jiang et al., 2020).

We summarize some key benefits that synthetic trajectory generation afford our method.

- **Consistency.** Because SO methods find a point within the feasible region at each step, the resulting constraint set is guaranteed to be consistent and conflict-free. This avoids distributional assumptions and conflict-resolution procedures that arise when preferences are elicited over a fixed trajectory dataset. Essentially, we never “ask the same (preference) question twice”.
- **Complexity guarantees.** The convergence bounds of SO methods are predicated on the volume removed from the feasible polyhedron at each step—a quantity we can guarantee when generating trajectories synthetically.
- **Controllability.** Trajectories for preference elicitation can be modified before presentation. For example, numerical outcomes can be rounded to improve human readability, or trajectories can be shifted toward a target distribution to make comparisons easier for non-expert evaluators who may struggle with out-of-distribution hypotheticals.

4 Conclusion

We have presented a framework that takes a task description and produces a human-aligned linear reward function, along with formalizations and algorithms for three contribution steps: a guided workflow for deriving outcome variables, a reduction of reward term selection to minimum-cost partial cover (solved via max-flow), and a geometric framing of weight fitting as a convex feasibility problem narrowed by preference queries. These steps address three recurring failure modes in reward design: redundancy (mitigated by causal cover), reward hacking (mitigated by grounding reward terms in elicited objectives rather than shaped intermediate behaviors), and preference misalignment (mitigated by producing a consistent feasible weight region by construction). We hope this framework serves as a principled foundation for further work on automated reward design and can be a step towards an interactive system that enables non-RL practitioners to design and iterate on reward functions.

References

- David S. Atkinson and Pravin M. Vaidya. A cutting plane algorithm for convex programming that uses analytic centers. *Mathematical Programming*, 69(1–3):1–43, July 1995. ISSN 0025-5610.
- André Barreto, Will Dabney, Rémi Munos, Jonathan J. Hunt, Tom Schaul, Hado van Hasselt, and David Silver. Successor features for transfer in reinforcement learning. In *Advances in Neural Information Processing Systems*, volume 30. Curran Associates, Inc., 2017.
- Serena Booth, W. Bradley Knox, Julie Shah, Scott Niekum, Peter Stone, and Alessandro Allievi. The perils of trial-and-error reward design: Misdesign through overfitting and invalid task specifications. In *Proceedings of the AAAI Conference on Artificial Intelligence*, volume 37, pp. 5920–5929, 2023.
- Paul Christiano, Jan Leike, Tom B. Brown, Miljan Martic, Shane Legg, and Dario Amodei. Deep reinforcement learning from human preferences. In *Advances in Neural Information Processing Systems*, volume 30. Curran Associates, Inc., 2017.
- E. A. Dinic. Algorithm for solution of a problem of maximum flow in a network with power estimation. *Soviet Mathematics Doklady*, 11:1277–1280, 1970.
- Gabriel Dulac-Arnold, Daniel Mankowitz, and Todd Hester. Challenges of real-world reinforcement learning, 2019. URL <https://arxiv.org/abs/1904.12901>.
- Jack Edmonds and Richard M. Karp. Theoretical improvements in algorithmic efficiency for network flow problems. *Journal of the ACM*, 19(2):248–264, 1972.
- Lester R. Ford and Delbert R. Fulkerson. Maximal flow through a network. *Canadian Journal of Mathematics*, 8:399–404, 1956. DOI: 10.4153/CJM-1956-045-5.
- M. Grötschel, L. Lovász, and A. Schrijver. The ellipsoid method and its consequences in combinatorial optimization. *Combinatorica*, 1(2):169–197, June 1981.
- Stephane Hatgis-Kessell. Specifying reward functions for RL without environment sampling. Manuscript under review, 2026.
- Haotian Jiang, Yin Tat Lee, Zhao Song, and Sam Chiu-wai Wong. An improved cutting plane method for convex optimization, convex-concave games, and its applications. In *Proceedings of the 52nd Annual ACM SIGACT Symposium on Theory of Computing*, STOC 2020, pp. 944–953, New York, NY, USA, 2020. Association for Computing Machinery. ISBN 9781450369794. DOI: 10.1145/3357713.3384284. URL <https://doi.org/10.1145/3357713.3384284>.
- Ralph L. Keeney. *Value-Focused Thinking: A Path to Creative Decisionmaking*. Harvard University Press, Cambridge, MA, 1992. ISBN 0-674-93197-1.
- W. Bradley Knox, Alessandro Allievi, Holger Banzhaf, Felix Schmitt, and Peter Stone. Reward (mis)design for autonomous driving. *Artificial Intelligence*, 316:103829, 2023. ISSN 0004-3702. DOI: 10.1016/j.artint.2022.103829. URL <https://www.sciencedirect.com/science/article/pii/S0004370222001692>.
- Andrew Y. Ng and Stuart J. Russell. Algorithms for inverse reinforcement learning. In *Proceedings of the Seventeenth International Conference on Machine Learning*, ICML ’00, pp. 663–670, San Francisco, CA, USA, 2000. Morgan Kaufmann Publishers Inc. ISBN 1558607072.
- Andrew Y. Ng, Daishi Harada, and Stuart Russell. Policy invariance under reward transformations: Theory and application to reward shaping. In *Proceedings of the Sixteenth International Conference on Machine Learning*, ICML ’99, pp. 278–287. Morgan Kaufmann Publishers Inc., 1999.

-
- Long Ouyang, Jeff Wu, Xu Jiang, Diogo Almeida, Carroll L. Wainwright, Pamela Mishkin, Chong Zhang, Sandhini Agarwal, Katarina Slama, Alex Ray, John Schulman, Jacob Hilton, Fraser Kelton, Luke Miller, Maddie Simens, Amanda Askell, Peter Welinder, Paul Christiano, Jan Leike, and Ryan Lowe. Training language models to follow instructions with human feedback, 2022. URL <https://arxiv.org/abs/2203.02155>.
- Dorsa Sadigh, Anca D. Dragan, S. Shankar Sastry, and Sanjit A. Seshia. Active preference-based learning of reward functions. In *Proceedings of Robotics: Science and Systems*, 2017. DOI: 10.15607/RSS.2017.XIII.053.
- Richard S. Sutton. The bitter lesson. <http://www.incompleteideas.net/IncIdeas/BitterLesson.html>, March 2019.
- Pravin M. Vaidya. A new algorithm for minimizing convex functions over convex sets. *Mathematical Programming*, 73(3):291–341, June 1996.
- Eric Wiewiora, Garrison W. Cottrell, and Charles Elkan. Principled methods for advising reinforcement learning agents. In *Proceedings of the 20th International Conference on Machine Learning*, pp. 792–799, 2003.

A Reduction of Minimum Cost Partial Cover

A.1 Preliminaries: Flow Networks (Ford & Fulkerson, 1956)

A *flow network* is a directed graph $\mathcal{N} = (V', E', \sigma, \tau)$ where each edge $e \in E'$ has a non-negative capacity $\text{cap}(e) \in \mathbb{Q}_{\geq 0} \cup \{\infty\}$, and two distinguished nodes play special roles.

Definition 3 (Supersource and Supersink). The supersource $\sigma \in V'$ is a node with no incoming edges, representing the origin of all flow. The supersink $\tau \in V'$ is a node with no outgoing edges, representing the destination of all flow.

Definition 4 (Flow). A flow $f : E' \rightarrow \mathbb{R}_{\geq 0}$ is an assignment of values to edges satisfying: (i) capacity constraints: $f(e) \leq \text{cap}(e)$ for all $e \in E'$, and (ii) conservation: for every node $v \notin \{\sigma, \tau\}$, the total flow into v equals the total flow out of v . The value of a flow is the total flow out of σ .

Definition 5 (s - t Cut). A σ - τ cut is a partition (A, B) of V' with $\sigma \in A$ and $\tau \in B$. Its capacity is the total capacity of edges from A to B :

$$\text{cap}(A, B) = \sum_{\substack{e=(u,v) \\ u \in A, v \in B}} \text{cap}(e).$$

A minimum cut is a σ - τ cut of minimum capacity.

Definition 6 (Residual Graph). Given a flow f in $\mathcal{N}$, the residual graph $\mathcal{N}_f$ has the same node set V' and contains, for each edge $e = (u, v) \in E'$: a forward edge (u, v) with residual capacity $\text{cap}(e) - f(e)$ if $f(e) < \text{cap}(e)$, and a backward edge (v, u) with residual capacity $f(e)$ if $f(e) > 0$.

Theorem 1 (Max-Flow Min-Cut). In any flow network, the maximum value of a σ - τ flow equals the minimum capacity of a σ - τ cut. Furthermore, a flow f^* is maximum if and only if τ is not reachable from σ in the residual graph $\mathcal{N}_{f^*}$. In this case, setting A to be the nodes reachable from σ in $\mathcal{N}_{f^*}$ and $B = V' \setminus A$ yields a minimum cut (A, B) .

A.2 Structural Characterization

We first show that valid covers are exactly the sets that hit every source-to-demand path. Let $\text{src}(G)$ denote the set of source nodes of G (nodes with in-degree zero).

Lemma 1. A set $S \subseteq V$ is a valid cover of D if and only if S intersects every directed path from any source $r \in \text{src}(G)$ to any demand node $d \in D$.

Proof. ($\Rightarrow$) Suppose S is a valid cover and let $P = v_0 \rightarrow v_1 \rightarrow \dots \rightarrow v_k$ be a directed path with $v_0 \in \text{src}(G)$ and $v_k \in D$. Suppose for contradiction that $P \cap S = \emptyset$. Since v_0 is a source it has no parents, so it is covered only if $v_0 \in S$ — a contradiction. Hence S must hit every such path.

($\Leftarrow$) Suppose S hits every source-to-demand path. We show every $d \in D$ is covered by induction on the length of the longest source-to- d path. For the base case, if d has no parents then $d \in \text{src}(G)$, and the path of length zero from d to itself must be hit by S , so $d \in S$ and d is covered.

For the inductive step, if $d \notin S$ then consider any parent u of d . Every directed path from a source to u can be extended by the edge $u \rightarrow d$ to a source-to- d path, which S must hit. Since S does not hit $u \rightarrow d$ at d (as $d \notin S$), it must hit the path at or before u . Hence every source-to- u path is hit by S . By the inductive hypothesis u is covered. Since u was arbitrary, every parent of d is covered, so d is covered. $\square$

A.3 Reduction to Min-Cut Proof

Theorem 2. The minimum cost partial cover of D equals the minimum σ - τ cut in $\mathcal{N}$, and hence equals the maximum σ - τ flow in $\mathcal{N}$ by the max-flow min-cut theorem.

Proof. Every finite-capacity edge in $\mathcal{N}$ has the form $v^{in} \rightarrow v^{out}$ for some $v \in V$, with capacity $c(v)$. A σ - τ cut of finite capacity must consist entirely of such edges. Let $S = \{v \in V : (v^{in} \rightarrow v^{out}) \in \text{cut}\}$ be the corresponding set of nodes.

Every directed path from σ to τ in $\mathcal{N}$ has the form

$$\sigma \rightarrow r^{in} \rightarrow r^{out} \rightarrow \dots \rightarrow d^{in} \rightarrow d^{out} \rightarrow \tau$$

for some $r \in \text{src}(G)$ and $d \in D$, and corresponds exactly to a source-to-demand path in G . The cut severs all such paths iff S hits every source-to-demand path in G . By Lemma 1 this is exactly the condition for S to be a valid cover. The cut capacity equals $\sum_{v \in S} c(v)$, so the minimum cut corresponds to the minimum cost valid cover. $\square$